

Springer Series in Statistics

Trevor Hastie
Robert Tibshirani
Jerome Friedman

The Elements of Statistical Learning

Data Mining, Inference, and Prediction

Second Edition

 Springer

Corrected 12th printing - Jan 13, 2017

To our parents:

Valerie and Patrick Hastie

Vera and Sami Tibshirani

Florence and Harry Friedman

and to our families:

Samantha, Timothy, and Lynda

Charlie, Ryan, Julie, and Cheryl

Melanie, Dora, Monika, and Ildiko

Preface to the Second Edition

In God we trust, all others bring data.

—William Edwards Deming (1900-1993)¹

We have been gratified by the popularity of the first edition of *The Elements of Statistical Learning*. This, along with the fast pace of research in the statistical learning field, motivated us to update our book with a second edition.

We have added four new chapters and updated some of the existing chapters. Because many readers are familiar with the layout of the first edition, we have tried to change it as little as possible. Here is a summary of the main changes:

¹On the Web, this quote has been widely attributed to both Deming and Robert W. Hayden; however Professor Hayden told us that he can claim no credit for this quote, and ironically we could find no “data” confirming that Deming actually said this.

Chapter	What's new
1. Introduction	
2. Overview of Supervised Learning	
3. Linear Methods for Regression	LAR algorithm and generalizations of the lasso
4. Linear Methods for Classification	Lasso path for logistic regression
5. Basis Expansions and Regularization	Additional illustrations of RKHS
6. Kernel Smoothing Methods	
7. Model Assessment and Selection	Strengths and pitfalls of cross-validation
8. Model Inference and Averaging	
9. Additive Models, Trees, and Related Methods	
10. Boosting and Additive Trees	New example from ecology; some material split off to Chapter 16.
11. Neural Networks	Bayesian neural nets and the NIPS 2003 challenge
12. Support Vector Machines and Flexible Discriminants	Path algorithm for SVM classifier
13. Prototype Methods and Nearest-Neighbors	
14. Unsupervised Learning	Spectral clustering, kernel PCA, sparse PCA, non-negative matrix factorization, archetypal analysis, nonlinear dimension reduction, Google page rank algorithm, a direct approach to ICA
15. Random Forests	New
16. Ensemble Learning	New
17. Undirected Graphical Models	New
18. High-Dimensional Problems	New

Some further notes:

- Our first edition was unfriendly to colorblind readers; in particular, we tended to favor red/green contrasts which are particularly troublesome. We have changed the color palette in this edition to a large extent, replacing the above with an orange/blue contrast.
- We have changed the name of Chapter 6 from “Kernel Methods” to “Kernel Smoothing Methods”, to avoid confusion with the machine-learning kernel method that is discussed in the context of support vector machines (Chapter 12) and more generally in Chapters 5 and 14.
- In the first edition, the discussion of error-rate estimation in Chapter 7 was sloppy, as we did not clearly differentiate the notions of conditional error rates (conditional on the training set) and unconditional rates. We have fixed this in the new edition.

- Chapters 15 and 16 follow naturally from Chapter 10, and the chapters are probably best read in that order.
- In Chapter 17, we have not attempted a comprehensive treatment of graphical models, and discuss only undirected models and some new methods for their estimation. Due to a lack of space, we have specifically omitted coverage of directed graphical models.
- Chapter 18 explores the “ $p \gg N$ ” problem, which is learning in high-dimensional feature spaces. These problems arise in many areas, including genomic and proteomic studies, and document classification.

We thank the many readers who have found the (too numerous) errors in the first edition. We apologize for those and have done our best to avoid errors in this new edition. We thank Mark Segal, Bala Rajaratnam, and Larry Wasserman for comments on some of the new chapters, and many Stanford graduate and post-doctoral students who offered comments, in particular Mohammed AlQuraishi, John Boik, Holger Hoeffling, Arian Maleki, Donal McMahon, Saharon Rosset, Babak Shababa, Daniela Witten, Ji Zhu and Hui Zou. We thank John Kimmel for his patience in guiding us through this new edition. RT dedicates this edition to the memory of Anna McPhee.

Trevor Hastie
Robert Tibshirani
Jerome Friedman

Stanford, California
 August 2008

Preface to the First Edition

We are drowning in information and starving for knowledge.

—Rutherford D. Roger

The field of Statistics is constantly challenged by the problems that science and industry brings to its door. In the early days, these problems often came from agricultural and industrial experiments and were relatively small in scope. With the advent of computers and the information age, statistical problems have exploded both in size and complexity. Challenges in the areas of data storage, organization and searching have led to the new field of “data mining”; statistical and computational problems in biology and medicine have created “bioinformatics.” Vast amounts of data are being generated in many fields, and the statistician’s job is to make sense of it all: to extract important patterns and trends, and understand “what the data says.” We call this *learning from data*.

The challenges in learning from data have led to a revolution in the statistical sciences. Since computation plays such a key role, it is not surprising that much of this new development has been done by researchers in other fields such as computer science and engineering.

The learning problems that we consider can be roughly categorized as either *supervised* or *unsupervised*. In supervised learning, the goal is to predict the value of an outcome measure based on a number of input measures; in unsupervised learning, there is no outcome measure, and the goal is to describe the associations and patterns among a set of input measures.

This book is our attempt to bring together many of the important new ideas in learning, and explain them in a statistical framework. While some mathematical details are needed, we emphasize the methods and their conceptual underpinnings rather than their theoretical properties. As a result, we hope that this book will appeal not just to statisticians but also to researchers and practitioners in a wide variety of fields.

Just as we have learned a great deal from researchers outside of the field of statistics, our statistical viewpoint may help others to better understand different aspects of learning:

There is no true interpretation of anything; interpretation is a vehicle in the service of human comprehension. The value of interpretation is in enabling others to fruitfully think about an idea.

—Andreas Buja

We would like to acknowledge the contribution of many people to the conception and completion of this book. David Andrews, Leo Breiman, Andreas Buja, John Chambers, Bradley Efron, Geoffrey Hinton, Werner Stuetzle, and John Tukey have greatly influenced our careers. Balasubramanian Narasimhan gave us advice and help on many computational problems, and maintained an excellent computing environment. Shin-Ho Bang helped in the production of a number of the figures. Lee Wilkinson gave valuable tips on color production. Ilana Belitskaya, Eva Cantoni, Maya Gupta, Michael Jordan, Shanti Gopatam, Radford Neal, Jorge Picazo, Bogdan Popescu, Olivier Renaud, Saharon Rosset, John Storey, Ji Zhu, Mu Zhu, two reviewers and many students read parts of the manuscript and offered helpful suggestions. John Kimmel was supportive, patient and helpful at every phase; MaryAnn Brickner and Frank Ganz headed a superb production team at Springer. Trevor Hastie would like to thank the statistics department at the University of Cape Town for their hospitality during the final stages of this book. We gratefully acknowledge NSF and NIH for their support of this work. Finally, we would like to thank our families and our parents for their love and support.

*Trevor Hastie
Robert Tibshirani
Jerome Friedman*

Stanford, California
May 2001

The quiet statisticians have changed our world; not by discovering new facts or technical developments, but by changing the ways that we reason, experiment and form our opinions

—Ian Hacking

Contents

Preface to the Second Edition	vii
Preface to the First Edition	xi
1 Introduction	1
2 Overview of Supervised Learning	9
2.1 Introduction	9
2.2 Variable Types and Terminology	9
2.3 Two Simple Approaches to Prediction:	
Least Squares and Nearest Neighbors	11
2.3.1 Linear Models and Least Squares	11
2.3.2 Nearest-Neighbor Methods	14
2.3.3 From Least Squares to Nearest Neighbors	16
2.4 Statistical Decision Theory	18
2.5 Local Methods in High Dimensions	22
2.6 Statistical Models, Supervised Learning	
and Function Approximation	28
2.6.1 A Statistical Model	
for the Joint Distribution $\Pr(X, Y)$	28
2.6.2 Supervised Learning	29
2.6.3 Function Approximation	29
2.7 Structured Regression Models	32
2.7.1 Difficulty of the Problem	32

2.8	Classes of Restricted Estimators	33
2.8.1	Roughness Penalty and Bayesian Methods . . .	34
2.8.2	Kernel Methods and Local Regression	34
2.8.3	Basis Functions and Dictionary Methods	35
2.9	Model Selection and the Bias–Variance Tradeoff	37
	Bibliographic Notes	39
	Exercises	39
3	Linear Methods for Regression	43
3.1	Introduction	43
3.2	Linear Regression Models and Least Squares	44
3.2.1	Example: Prostate Cancer	49
3.2.2	The Gauss–Markov Theorem	51
3.2.3	Multiple Regression from Simple Univariate Regression	52
3.2.4	Multiple Outputs	56
3.3	Subset Selection	57
3.3.1	Best-Subset Selection	57
3.3.2	Forward- and Backward-Stepwise Selection . . .	58
3.3.3	Forward-Stagewise Regression	60
3.3.4	Prostate Cancer Data Example (Continued) . .	61
3.4	Shrinkage Methods	61
3.4.1	Ridge Regression	61
3.4.2	The Lasso	68
3.4.3	Discussion: Subset Selection, Ridge Regression and the Lasso	69
3.4.4	Least Angle Regression	73
3.5	Methods Using Derived Input Directions	79
3.5.1	Principal Components Regression	79
3.5.2	Partial Least Squares	80
3.6	Discussion: A Comparison of the Selection and Shrinkage Methods	82
3.7	Multiple Outcome Shrinkage and Selection	84
3.8	More on the Lasso and Related Path Algorithms	86
3.8.1	Incremental Forward Stagewise Regression . . .	86
3.8.2	Piecewise-Linear Path Algorithms	89
3.8.3	The Dantzig Selector	89
3.8.4	The Grouped Lasso	90
3.8.5	Further Properties of the Lasso	91
3.8.6	Pathwise Coordinate Optimization	92
3.9	Computational Considerations	93
	Bibliographic Notes	94
	Exercises	94

4	Linear Methods for Classification	101
4.1	Introduction	101
4.2	Linear Regression of an Indicator Matrix	103
4.3	Linear Discriminant Analysis	106
4.3.1	Regularized Discriminant Analysis	112
4.3.2	Computations for LDA	113
4.3.3	Reduced-Rank Linear Discriminant Analysis	113
4.4	Logistic Regression	119
4.4.1	Fitting Logistic Regression Models	120
4.4.2	Example: South African Heart Disease	122
4.4.3	Quadratic Approximations and Inference	124
4.4.4	L_1 Regularized Logistic Regression	125
4.4.5	Logistic Regression or LDA?	127
4.5	Separating Hyperplanes	129
4.5.1	Rosenblatt's Perceptron Learning Algorithm	130
4.5.2	Optimal Separating Hyperplanes	132
	Bibliographic Notes	135
	Exercises	135
5	Basis Expansions and Regularization	139
5.1	Introduction	139
5.2	Piecewise Polynomials and Splines	141
5.2.1	Natural Cubic Splines	144
5.2.2	Example: South African Heart Disease (Continued)	146
5.2.3	Example: Phoneme Recognition	148
5.3	Filtering and Feature Extraction	150
5.4	Smoothing Splines	151
5.4.1	Degrees of Freedom and Smoother Matrices	153
5.5	Automatic Selection of the Smoothing Parameters	156
5.5.1	Fixing the Degrees of Freedom	158
5.5.2	The Bias–Variance Tradeoff	158
5.6	Nonparametric Logistic Regression	161
5.7	Multidimensional Splines	162
5.8	Regularization and Reproducing Kernel Hilbert Spaces	167
5.8.1	Spaces of Functions Generated by Kernels	168
5.8.2	Examples of RKHS	170
5.9	Wavelet Smoothing	174
5.9.1	Wavelet Bases and the Wavelet Transform	176
5.9.2	Adaptive Wavelet Filtering	179
	Bibliographic Notes	181
	Exercises	181
	Appendix: Computational Considerations for Splines	186
	Appendix: B -splines	186
	Appendix: Computations for Smoothing Splines	189

6	Kernel Smoothing Methods	191
6.1	One-Dimensional Kernel Smoothers	192
6.1.1	Local Linear Regression	194
6.1.2	Local Polynomial Regression	197
6.2	Selecting the Width of the Kernel	198
6.3	Local Regression in $\mathbb{R}^p$	200
6.4	Structured Local Regression Models in $\mathbb{R}^p$	201
6.4.1	Structured Kernels	203
6.4.2	Structured Regression Functions	203
6.5	Local Likelihood and Other Models	205
6.6	Kernel Density Estimation and Classification	208
6.6.1	Kernel Density Estimation	208
6.6.2	Kernel Density Classification	210
6.6.3	The Naive Bayes Classifier	210
6.7	Radial Basis Functions and Kernels	212
6.8	Mixture Models for Density Estimation and Classification	214
6.9	Computational Considerations	216
	Bibliographic Notes	216
	Exercises	216
7	Model Assessment and Selection	219
7.1	Introduction	219
7.2	Bias, Variance and Model Complexity	219
7.3	The Bias–Variance Decomposition	223
7.3.1	Example: Bias–Variance Tradeoff	226
7.4	Optimism of the Training Error Rate	228
7.5	Estimates of In-Sample Prediction Error	230
7.6	The Effective Number of Parameters	232
7.7	The Bayesian Approach and BIC	233
7.8	Minimum Description Length	235
7.9	Vapnik–Chervonenkis Dimension	237
7.9.1	Example (Continued)	239
7.10	Cross-Validation	241
7.10.1	K -Fold Cross-Validation	241
7.10.2	The Wrong and Right Way to Do Cross-validation	245
7.10.3	Does Cross-Validation Really Work?	247
7.11	Bootstrap Methods	249
7.11.1	Example (Continued)	252
7.12	Conditional or Expected Test Error?	254
	Bibliographic Notes	257
	Exercises	257
8	Model Inference and Averaging	261
8.1	Introduction	261

8.2	The Bootstrap and Maximum Likelihood Methods	261
8.2.1	A Smoothing Example	261
8.2.2	Maximum Likelihood Inference	265
8.2.3	Bootstrap versus Maximum Likelihood	267
8.3	Bayesian Methods	267
8.4	Relationship Between the Bootstrap and Bayesian Inference	271
8.5	The EM Algorithm	272
8.5.1	Two-Component Mixture Model	272
8.5.2	The EM Algorithm in General	276
8.5.3	EM as a Maximization–Maximization Procedure	277
8.6	MCMC for Sampling from the Posterior	279
8.7	Bagging	282
8.7.1	Example: Trees with Simulated Data	283
8.8	Model Averaging and Stacking	288
8.9	Stochastic Search: Bumping	290
	Bibliographic Notes	292
	Exercises	293
9	Additive Models, Trees, and Related Methods	295
9.1	Generalized Additive Models	295
9.1.1	Fitting Additive Models	297
9.1.2	Example: Additive Logistic Regression	299
9.1.3	Summary	304
9.2	Tree-Based Methods	305
9.2.1	Background	305
9.2.2	Regression Trees	307
9.2.3	Classification Trees	308
9.2.4	Other Issues	310
9.2.5	Spam Example (Continued)	313
9.3	PRIM: Bump Hunting	317
9.3.1	Spam Example (Continued)	320
9.4	MARS: Multivariate Adaptive Regression Splines	321
9.4.1	Spam Example (Continued)	326
9.4.2	Example (Simulated Data)	327
9.4.3	Other Issues	328
9.5	Hierarchical Mixtures of Experts	329
9.6	Missing Data	332
9.7	Computational Considerations	334
	Bibliographic Notes	334
	Exercises	335
10	Boosting and Additive Trees	337
10.1	Boosting Methods	337
10.1.1	Outline of This Chapter	340

10.2	Boosting Fits an Additive Model	341
10.3	Forward Stagewise Additive Modeling	342
10.4	Exponential Loss and AdaBoost	343
10.5	Why Exponential Loss?	345
10.6	Loss Functions and Robustness	346
10.7	“Off-the-Shelf” Procedures for Data Mining	350
10.8	Example: Spam Data	352
10.9	Boosting Trees	353
10.10	Numerical Optimization via Gradient Boosting	358
10.10.1	Steepest Descent	358
10.10.2	Gradient Boosting	359
10.10.3	Implementations of Gradient Boosting	360
10.11	Right-Sized Trees for Boosting	361
10.12	Regularization	364
10.12.1	Shrinkage	364
10.12.2	Subsampling	365
10.13	Interpretation	367
10.13.1	Relative Importance of Predictor Variables	367
10.13.2	Partial Dependence Plots	369
10.14	Illustrations	371
10.14.1	California Housing	371
10.14.2	New Zealand Fish	375
10.14.3	Demographics Data	379
	Bibliographic Notes	380
	Exercises	384
11	Neural Networks	389
11.1	Introduction	389
11.2	Projection Pursuit Regression	389
11.3	Neural Networks	392
11.4	Fitting Neural Networks	395
11.5	Some Issues in Training Neural Networks	397
11.5.1	Starting Values	397
11.5.2	Overfitting	398
11.5.3	Scaling of the Inputs	398
11.5.4	Number of Hidden Units and Layers	400
11.5.5	Multiple Minima	400
11.6	Example: Simulated Data	401
11.7	Example: ZIP Code Data	404
11.8	Discussion	408
11.9	Bayesian Neural Nets and the NIPS 2003 Challenge	409
11.9.1	Bayes, Boosting and Bagging	410
11.9.2	Performance Comparisons	412
11.10	Computational Considerations	414
	Bibliographic Notes	415

Exercises	415
12 Support Vector Machines and Flexible Discriminants	417
12.1 Introduction	417
12.2 The Support Vector Classifier	417
12.2.1 Computing the Support Vector Classifier	420
12.2.2 Mixture Example (Continued)	421
12.3 Support Vector Machines and Kernels	423
12.3.1 Computing the SVM for Classification	423
12.3.2 The SVM as a Penalization Method	426
12.3.3 Function Estimation and Reproducing Kernels	428
12.3.4 SVMs and the Curse of Dimensionality	431
12.3.5 A Path Algorithm for the SVM Classifier	432
12.3.6 Support Vector Machines for Regression	434
12.3.7 Regression and Kernels	436
12.3.8 Discussion	438
12.4 Generalizing Linear Discriminant Analysis	438
12.5 Flexible Discriminant Analysis	440
12.5.1 Computing the FDA Estimates	444
12.6 Penalized Discriminant Analysis	446
12.7 Mixture Discriminant Analysis	449
12.7.1 Example: Waveform Data	451
Bibliographic Notes	455
Exercises	455
13 Prototype Methods and Nearest-Neighbors	459
13.1 Introduction	459
13.2 Prototype Methods	459
13.2.1 K -means Clustering	460
13.2.2 Learning Vector Quantization	462
13.2.3 Gaussian Mixtures	463
13.3 k -Nearest-Neighbor Classifiers	463
13.3.1 Example: A Comparative Study	468
13.3.2 Example: k -Nearest-Neighbors and Image Scene Classification	470
13.3.3 Invariant Metrics and Tangent Distance	471
13.4 Adaptive Nearest-Neighbor Methods	475
13.4.1 Example	478
13.4.2 Global Dimension Reduction for Nearest-Neighbors	479
13.5 Computational Considerations	480
Bibliographic Notes	481
Exercises	481

14	Unsupervised Learning	485
14.1	Introduction	485
14.2	Association Rules	487
14.2.1	Market Basket Analysis	488
14.2.2	The Apriori Algorithm	489
14.2.3	Example: Market Basket Analysis	492
14.2.4	Unsupervised as Supervised Learning	495
14.2.5	Generalized Association Rules	497
14.2.6	Choice of Supervised Learning Method	499
14.2.7	Example: Market Basket Analysis (Continued)	499
14.3	Cluster Analysis	501
14.3.1	Proximity Matrices	503
14.3.2	Dissimilarities Based on Attributes	503
14.3.3	Object Dissimilarity	505
14.3.4	Clustering Algorithms	507
14.3.5	Combinatorial Algorithms	507
14.3.6	K -means	509
14.3.7	Gaussian Mixtures as Soft K -means Clustering	510
14.3.8	Example: Human Tumor Microarray Data	512
14.3.9	Vector Quantization	514
14.3.10	K -medoids	515
14.3.11	Practical Issues	518
14.3.12	Hierarchical Clustering	520
14.4	Self-Organizing Maps	528
14.5	Principal Components, Curves and Surfaces	534
14.5.1	Principal Components	534
14.5.2	Principal Curves and Surfaces	541
14.5.3	Spectral Clustering	544
14.5.4	Kernel Principal Components	547
14.5.5	Sparse Principal Components	550
14.6	Non-negative Matrix Factorization	553
14.6.1	Archetypal Analysis	554
14.7	Independent Component Analysis and Exploratory Projection Pursuit	557
14.7.1	Latent Variables and Factor Analysis	558
14.7.2	Independent Component Analysis	560
14.7.3	Exploratory Projection Pursuit	565
14.7.4	A Direct Approach to ICA	565
14.8	Multidimensional Scaling	570
14.9	Nonlinear Dimension Reduction and Local Multidimensional Scaling	572
14.10	The Google PageRank Algorithm	576
	Bibliographic Notes	578
	Exercises	579

15 Random Forests	587
15.1 Introduction	587
15.2 Definition of Random Forests	587
15.3 Details of Random Forests	592
15.3.1 Out of Bag Samples	592
15.3.2 Variable Importance	593
15.3.3 Proximity Plots	595
15.3.4 Random Forests and Overfitting	596
15.4 Analysis of Random Forests	597
15.4.1 Variance and the De-Correlation Effect	597
15.4.2 Bias	600
15.4.3 Adaptive Nearest Neighbors	601
Bibliographic Notes	602
Exercises	603
16 Ensemble Learning	605
16.1 Introduction	605
16.2 Boosting and Regularization Paths	607
16.2.1 Penalized Regression	607
16.2.2 The “Bet on Sparsity” Principle	610
16.2.3 Regularization Paths, Over-fitting and Margins	613
16.3 Learning Ensembles	616
16.3.1 Learning a Good Ensemble	617
16.3.2 Rule Ensembles	622
Bibliographic Notes	623
Exercises	624
17 Undirected Graphical Models	625
17.1 Introduction	625
17.2 Markov Graphs and Their Properties	627
17.3 Undirected Graphical Models for Continuous Variables	630
17.3.1 Estimation of the Parameters when the Graph Structure is Known	631
17.3.2 Estimation of the Graph Structure	635
17.4 Undirected Graphical Models for Discrete Variables	638
17.4.1 Estimation of the Parameters when the Graph Structure is Known	639
17.4.2 Hidden Nodes	641
17.4.3 Estimation of the Graph Structure	642
17.4.4 Restricted Boltzmann Machines	643
Exercises	645
18 High-Dimensional Problems: $p \gg N$	649
18.1 When p is Much Bigger than N	649

18.2	Diagonal Linear Discriminant Analysis and Nearest Shrunken Centroids	651
18.3	Linear Classifiers with Quadratic Regularization	654
18.3.1	Regularized Discriminant Analysis	656
18.3.2	Logistic Regression with Quadratic Regularization	657
18.3.3	The Support Vector Classifier	657
18.3.4	Feature Selection	658
18.3.5	Computational Shortcuts When $p \gg N$	659
18.4	Linear Classifiers with L_1 Regularization	661
18.4.1	Application of Lasso to Protein Mass Spectroscopy	664
18.4.2	The Fused Lasso for Functional Data	666
18.5	Classification When Features are Unavailable	668
18.5.1	Example: String Kernels and Protein Classification	668
18.5.2	Classification and Other Models Using Inner-Product Kernels and Pairwise Distances	670
18.5.3	Example: Abstracts Classification	672
18.6	High-Dimensional Regression: Supervised Principal Components	674
18.6.1	Connection to Latent-Variable Modeling	678
18.6.2	Relationship with Partial Least Squares	680
18.6.3	Pre-Conditioning for Feature Selection	681
18.7	Feature Assessment and the Multiple-Testing Problem	683
18.7.1	The False Discovery Rate	687
18.7.2	Asymmetric Cutpoints and the SAM Procedure	690
18.7.3	A Bayesian Interpretation of the FDR	692
18.8	Bibliographic Notes	693
	Exercises	694
	References	699
	Author Index	729
	Index	737

1

Introduction

Statistical learning plays a key role in many areas of science, finance and industry. Here are some examples of learning problems:

- Predict whether a patient, hospitalized due to a heart attack, will have a second heart attack. The prediction is to be based on demographic, diet and clinical measurements for that patient.
- Predict the price of a stock in 6 months from now, on the basis of company performance measures and economic data.
- Identify the numbers in a handwritten ZIP code, from a digitized image.
- Estimate the amount of glucose in the blood of a diabetic person, from the infrared absorption spectrum of that person's blood.
- Identify the risk factors for prostate cancer, based on clinical and demographic variables.

The science of learning plays a key role in the fields of statistics, data mining and artificial intelligence, intersecting with areas of engineering and other disciplines.

This book is about learning from data. In a typical scenario, we have an outcome measurement, usually quantitative (such as a stock price) or categorical (such as heart attack/no heart attack), that we wish to predict based on a set of *features* (such as diet and clinical measurements). We have a *training set* of data, in which we observe the outcome and feature

TABLE 1.1. Average percentage of words or characters in an email message equal to the indicated word or character. We have chosen the words and characters showing the largest difference between **spam** and **email**.

	george	you	your	hp	free	hpl	!	our	re	edu	remove
spam	0.00	2.26	1.38	0.02	0.52	0.01	0.51	0.51	0.13	0.01	0.28
email	1.27	1.27	0.44	0.90	0.07	0.43	0.11	0.18	0.42	0.29	0.01

measurements for a set of objects (such as people). Using this data we build a prediction model, or *learner*, which will enable us to predict the outcome for new unseen objects. A good learner is one that accurately predicts such an outcome.

The examples above describe what is called the *supervised learning* problem. It is called “supervised” because of the presence of the outcome variable to guide the learning process. In the *unsupervised learning problem*, we observe only the features and have no measurements of the outcome. Our task is rather to describe how the data are organized or clustered. We devote most of this book to supervised learning; the unsupervised problem is less developed in the literature, and is the focus of Chapter 14.

Here are some examples of real learning problems that are discussed in this book.

Example 1: Email Spam

The data for this example consists of information from 4601 email messages, in a study to try to predict whether the email was junk email, or “spam.” The objective was to design an automatic spam detector that could filter out spam before clogging the users’ mailboxes. For all 4601 email messages, the true outcome (email type) **email** or **spam** is available, along with the relative frequencies of 57 of the most commonly occurring words and punctuation marks in the email message. This is a supervised learning problem, with the outcome the class variable **email/spam**. It is also called a *classification* problem.

Table 1.1 lists the words and characters showing the largest average difference between **spam** and **email**.

Our learning method has to decide which features to use and how: for example, we might use a rule such as

```
if (%george < 0.6) & (%you > 1.5) then spam
else email.
```

Another form of a rule might be:

```
if (0.2 · %you − 0.3 · %george) > 0 then spam
else email.
```

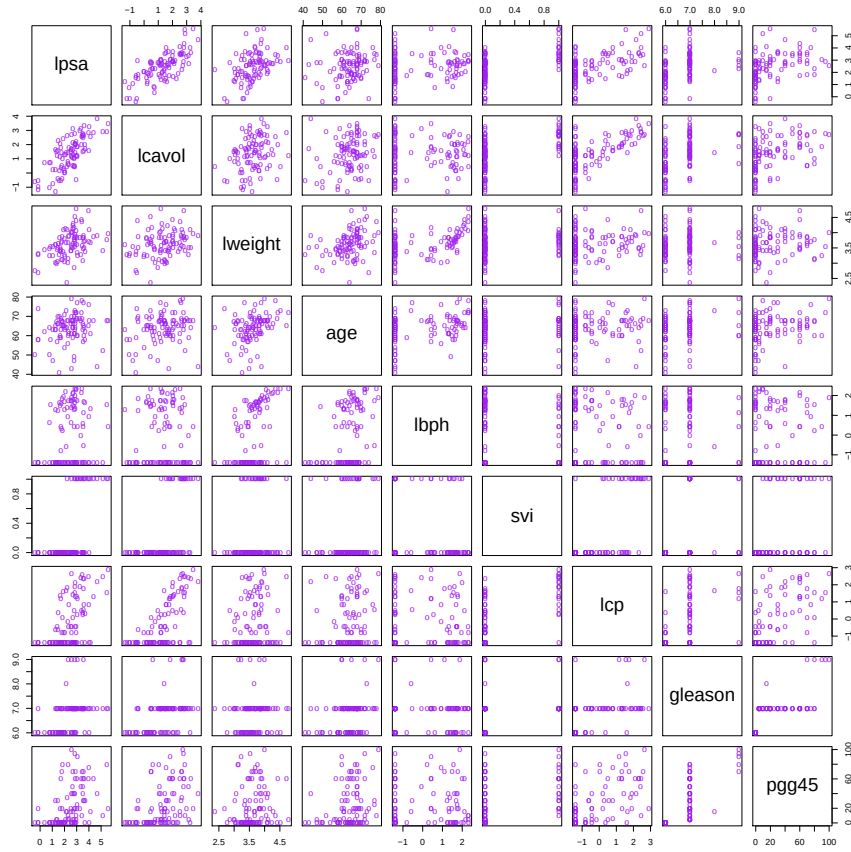


FIGURE 1.1. Scatterplot matrix of the prostate cancer data. The first row shows the response against each of the predictors in turn. Two of the predictors, *svi* and *gleason*, are categorical.

For this problem not all errors are equal; we want to avoid filtering out good email, while letting spam get through is not desirable but less serious in its consequences. We discuss a number of different methods for tackling this learning problem in the book.

Example 2: Prostate Cancer

The data for this example, displayed in Figure 1.1¹, come from a study by Stamey et al. (1989) that examined the correlation between the level of

¹There was an error in these data in the first edition of this book. Subject 32 had a value of 6.1 for *lweight*, which translates to a 449 gm prostate! The correct value is 44.9 gm. We are grateful to Prof. Stephen W. Link for alerting us to this error.

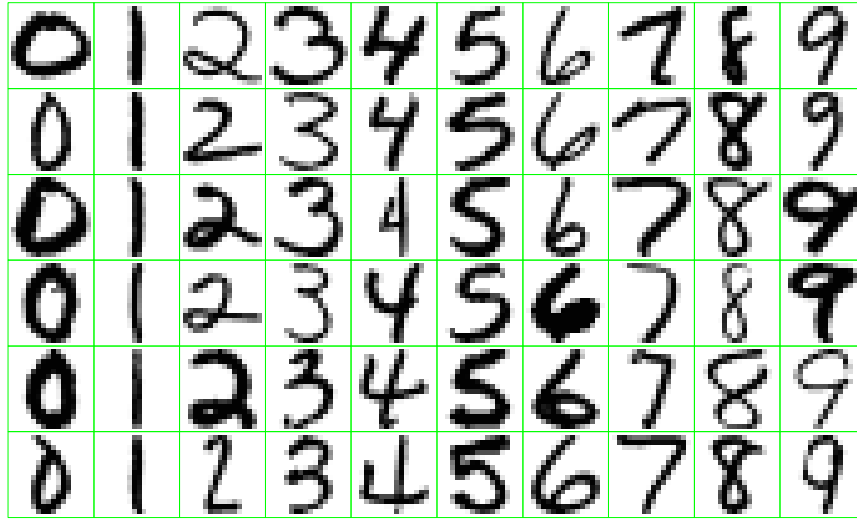


FIGURE 1.2. Examples of handwritten digits from U.S. postal envelopes.

prostate specific antigen (PSA) and a number of clinical measures, in 97 men who were about to receive a radical prostatectomy.

The goal is to predict the log of PSA (`lpsa`) from a number of measurements including log cancer volume (`lcavol`), log prostate weight `lweight`, age, log of benign prostatic hyperplasia amount `lbph`, seminal vesicle invasion `svi`, log of capsular penetration `lcp`, Gleason score `gleason`, and percent of Gleason scores 4 or 5 `pgg45`. Figure 1.1 is a scatterplot matrix of the variables. Some correlations with `lpsa` are evident, but a good predictive model is difficult to construct by eye.

This is a supervised learning problem, known as a *regression problem*, because the outcome measurement is quantitative.

Example 3: Handwritten Digit Recognition

The data from this example come from the handwritten ZIP codes on envelopes from U.S. postal mail. Each image is a segment from a five digit ZIP code, isolating a single digit. The images are 16×16 eight-bit grayscale maps, with each pixel ranging in intensity from 0 to 255. Some sample images are shown in Figure 1.2.

The images have been normalized to have approximately the same size and orientation. The task is to predict, from the 16×16 matrix of pixel intensities, the identity of each image (0, 1, ..., 9) quickly and accurately. If it is accurate enough, the resulting algorithm would be used as part of an automatic sorting procedure for envelopes. This is a classification problem for which the error rate needs to be kept very low to avoid misdirection of

mail. In order to achieve this low error rate, some objects can be assigned to a “don’t know” category, and sorted instead by hand.

Example 4: DNA Expression Microarrays

DNA stands for deoxyribonucleic acid, and is the basic material that makes up human chromosomes. DNA microarrays measure the expression of a gene in a cell by measuring the amount of mRNA (messenger ribonucleic acid) present for that gene. Microarrays are considered a breakthrough technology in biology, facilitating the quantitative study of thousands of genes simultaneously from a single sample of cells.

Here is how a DNA microarray works. The nucleotide sequences for a few thousand genes are printed on a glass slide. A target sample and a reference sample are labeled with red and green dyes, and each are hybridized with the DNA on the slide. Through fluoroscopy, the log (red/green) intensities of RNA hybridizing at each site is measured. The result is a few thousand numbers, typically ranging from say -6 to 6 , measuring the expression level of each gene in the target relative to the reference sample. Positive values indicate higher expression in the target versus the reference, and vice versa for negative values.

A gene expression dataset collects together the expression values from a series of DNA microarray experiments, with each column representing an experiment. There are therefore several thousand rows representing individual genes, and tens of columns representing samples: in the particular example of Figure 1.3 there are 6830 genes (rows) and 64 samples (columns), although for clarity only a random sample of 100 rows are shown. The figure displays the data set as a heat map, ranging from green (negative) to red (positive). The samples are 64 cancer tumors from different patients.

The challenge here is to understand how the genes and samples are organized. Typical questions include the following:

- (a) which samples are most similar to each other, in terms of their expression profiles across genes?
- (b) which genes are most similar to each other, in terms of their expression profiles across samples?
- (c) do certain genes show very high (or low) expression for certain cancer samples?

We could view this task as a regression problem, with two categorical predictor variables—genes and samples—with the response variable being the level of expression. However, it is probably more useful to view it as *unsupervised learning* problem. For example, for question (a) above, we think of the samples as points in 6830-dimensional space, which we want to *cluster* together in some way.

Who Should Read this Book

This book is designed for researchers and students in a broad variety of fields: statistics, artificial intelligence, engineering, finance and others. We expect that the reader will have had at least one elementary course in statistics, covering basic topics including linear regression.

We have not attempted to write a comprehensive catalog of learning methods, but rather to describe some of the most important techniques. Equally notable, we describe the underlying concepts and considerations by which a researcher can judge a learning method. We have tried to write this book in an intuitive fashion, emphasizing concepts rather than mathematical details.

As statisticians, our exposition will naturally reflect our backgrounds and areas of expertise. However in the past eight years we have been attending conferences in neural networks, data mining and machine learning, and our thinking has been heavily influenced by these exciting fields. This influence is evident in our current research, and in this book.

How This Book is Organized

Our view is that one must understand simple methods before trying to grasp more complex ones. Hence, after giving an overview of the supervising learning problem in [Chapter 2](#), we discuss linear methods for regression and classification in [Chapters 3 and 4](#). In [Chapter 5](#) we describe splines, wavelets and regularization/penalization methods for a single predictor, while [Chapter 6](#) covers kernel methods and local regression. Both of these sets of methods are important building blocks for high-dimensional learning techniques. Model assessment and selection is the topic of [Chapter 7](#), covering the concepts of bias and variance, overfitting and methods such as cross-validation for choosing models. [Chapter 8](#) discusses model inference and averaging, including an overview of maximum likelihood, Bayesian inference and the bootstrap, the EM algorithm, Gibbs sampling and bagging. A related procedure called boosting is the focus of [Chapter 10](#).

In [Chapters 9–13](#) we describe a series of structured methods for supervised learning, with [Chapters 9 and 11](#) covering regression and [Chapters 12 and 13](#) focusing on classification. [Chapter 14](#) describes methods for unsupervised learning. Two recently proposed techniques, random forests and ensemble learning, are discussed in [Chapters 15 and 16](#). We describe undirected graphical models in [Chapter 17](#) and finally we study high-dimensional problems in [Chapter 18](#).

At the end of each chapter we discuss [computational considerations](#) important for data mining applications, including how the computations scale with the number of observations and predictors. Each chapter ends with [Bibliographic Notes](#) giving background references for the material.

We recommend that Chapters 1–4 be first read in sequence. Chapter 7 should also be considered mandatory, as it covers central concepts that pertain to all learning methods. With this in mind, the rest of the book can be read sequentially, or sampled, depending on the reader’s interest.



The symbol  indicates a technically difficult section, one that can be skipped without interrupting the flow of the discussion.

Book Website

The website for this book is located at

`http://www-stat.stanford.edu/ElemStatLearn`

It contains a number of resources, including many of the datasets used in this book.

Note for Instructors

We have successively used the first edition of this book as the basis for a two-quarter course, and with the additional materials in this second edition, it could even be used for a three-quarter sequence. Exercises are provided at the end of each chapter. It is important for students to have access to good software tools for these topics. We used the R and S-PLUS programming languages in our courses.

2

Overview of Supervised Learning

2.1 Introduction

The first three examples described in Chapter 1 have several components in common. For each there is a set of variables that might be denoted as *inputs*, which are measured or preset. These have some influence on one or more *outputs*. For each example the goal is to use the inputs to predict the values of the outputs. This exercise is called *supervised learning*.

We have used the more modern language of machine learning. In the statistical literature the inputs are often called the *predictors*, a term we will use interchangeably with inputs, and more classically the *independent variables*. In the pattern recognition literature the term *features* is preferred, which we use as well. The outputs are called the *responses*, or classically the *dependent variables*.

2.2 Variable Types and Terminology

The outputs vary in nature among the examples. In the glucose prediction example, the output is a *quantitative* measurement, where some measurements are bigger than others, and measurements close in value are close in nature. In the famous Iris discrimination example due to R. A. Fisher, the output is *qualitative* (species of Iris) and assumes values in a finite set $\mathcal{G} = \{\textit{Virginica}, \textit{Setosa} \text{ and } \textit{Versicolor}\}$. In the handwritten digit example the output is one of 10 different digit *classes*: $\mathcal{G} = \{0, 1, \dots, 9\}$. In both of

these there is no explicit ordering in the classes, and in fact often descriptive labels rather than numbers are used to denote the classes. Qualitative variables are also referred to as *categorical* or *discrete* variables as well as *factors*.

For both types of outputs it makes sense to think of using the inputs to predict the output. Given some specific atmospheric measurements today and yesterday, we want to predict the ozone level tomorrow. Given the grayscale values for the pixels of the digitized image of the handwritten digit, we want to predict its class label.

This distinction in output type has led to a naming convention for the prediction tasks: *regression* when we predict quantitative outputs, and *classification* when we predict qualitative outputs. We will see that these two tasks have a lot in common, and in particular both can be viewed as a task in function approximation.

Inputs also vary in measurement type; we can have some of each of qualitative and quantitative input variables. These have also led to distinctions in the types of methods that are used for prediction: some methods are defined most naturally for quantitative inputs, some most naturally for qualitative and some for both.

A third variable type is *ordered categorical*, such as *small*, *medium* and *large*, where there is an ordering between the values, but no metric notion is appropriate (the difference between medium and small need not be the same as that between large and medium). These are discussed further in Chapter 4.

Qualitative variables are typically represented numerically by codes. The easiest case is when there are only two classes or categories, such as “success” or “failure,” “survived” or “died.” These are often represented by a single binary digit or bit as 0 or 1, or else by -1 and 1 . For reasons that will become apparent, such numeric codes are sometimes referred to as *targets*. When there are more than two categories, several alternatives are available. The most useful and commonly used coding is via *dummy variables*. Here a K -level qualitative variable is represented by a vector of K binary variables or bits, only one of which is “on” at a time. Although more compact coding schemes are possible, dummy variables are symmetric in the levels of the factor.

We will typically denote an input variable by the symbol X . If X is a vector, its components can be accessed by subscripts X_j . Quantitative outputs will be denoted by Y , and qualitative outputs by G (for group). We use uppercase letters such as X , Y or G when referring to the generic aspects of a variable. Observed values are written in lowercase; hence the i th observed value of X is written as x_i (where x_i is again a scalar or vector). Matrices are represented by bold uppercase letters; for example, a set of N input p -vectors x_i , $i = 1, \dots, N$ would be represented by the $N \times p$ matrix $\mathbf{X}$. In general, vectors will not be bold, except when they have N components; this convention distinguishes a p -vector of inputs x_i for the

i th observation from the N -vector $\mathbf{x}_j$ consisting of all the observations on variable X_j . Since all vectors are assumed to be column vectors, the i th row of $\mathbf{X}$ is x_i^T , the vector transpose of x_i .

For the moment we can loosely state the learning task as follows: given the value of an input vector X , make a good prediction of the output Y , denoted by $\hat{Y}$ (pronounced “y-hat”). If Y takes values in $\mathbb{R}$ then so should $\hat{Y}$; likewise for categorical outputs, $\hat{G}$ should take values in the same set $\mathcal{G}$ associated with G .

For a two-class G , one approach is to denote the binary coded target as Y , and then treat it as a quantitative output. The predictions $\hat{Y}$ will typically lie in $[0, 1]$, and we can assign to $\hat{G}$ the class label according to whether $\hat{y} > 0.5$. This approach generalizes to K -level qualitative outputs as well.

We need data to construct prediction rules, often a lot of it. We thus suppose we have available a set of measurements (x_i, y_i) or (x_i, g_i) , $i = 1, \dots, N$, known as the *training data*, with which to construct our prediction rule.

2.3 Two Simple Approaches to Prediction: Least Squares and Nearest Neighbors

In this section we develop two simple but powerful prediction methods: the linear model fit by least squares and the k -nearest-neighbor prediction rule. The linear model makes huge assumptions about structure and yields stable but possibly inaccurate predictions. The method of k -nearest neighbors makes very mild structural assumptions: its predictions are often accurate but can be unstable.

2.3.1 Linear Models and Least Squares

The linear model has been a mainstay of statistics for the past 30 years and remains one of our most important tools. Given a vector of inputs $X^T = (X_1, X_2, \dots, X_p)$, we predict the output Y via the model

$$\hat{Y} = \hat{\beta}_0 + \sum_{j=1}^p X_j \hat{\beta}_j. \quad (2.1)$$

The term $\hat{\beta}_0$ is the intercept, also known as the *bias* in machine learning. Often it is convenient to include the constant variable 1 in X , include $\hat{\beta}_0$ in the vector of coefficients $\hat{\beta}$, and then write the linear model in vector form as an inner product

$$\hat{Y} = X^T \hat{\beta}, \quad (2.2)$$

where X^T denotes vector or matrix transpose (X being a column vector). Here we are modeling a single output, so $\hat{Y}$ is a scalar; in general $\hat{Y}$ can be a K -vector, in which case β would be a $p \times K$ matrix of coefficients. In the $(p+1)$ -dimensional input-output space, $(X, \hat{Y})$ represents a hyperplane. If the constant is included in X , then the hyperplane includes the origin and is a subspace; if not, it is an affine set cutting the Y -axis at the point $(0, \hat{\beta}_0)$. From now on we assume that the intercept is included in $\hat{\beta}$.

Viewed as a function over the p -dimensional input space, $f(X) = X^T \beta$ is linear, and the gradient $f'(X) = \beta$ is a vector in input space that points in the steepest uphill direction.

How do we fit the linear model to a set of training data? There are many different methods, but by far the most popular is the method of *least squares*. In this approach, we pick the coefficients β to minimize the residual sum of squares

$$\text{RSS}(\beta) = \sum_{i=1}^N (y_i - x_i^T \beta)^2. \quad (2.3)$$

$\text{RSS}(\beta)$ is a quadratic function of the parameters, and hence its minimum always exists, but may not be unique. The solution is easiest to characterize in matrix notation. We can write

$$\text{RSS}(\beta) = (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta), \quad (2.4)$$

where $\mathbf{X}$ is an $N \times p$ matrix with each row an input vector, and $\mathbf{y}$ is an N -vector of the outputs in the training set. Differentiating w.r.t. β we get the *normal equations*

$$\mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta) = 0. \quad (2.5)$$

If $\mathbf{X}^T \mathbf{X}$ is nonsingular, then the unique solution is given by

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}, \quad (2.6)$$

and the fitted value at the i th input x_i is $\hat{y}_i = \hat{y}(x_i) = x_i^T \hat{\beta}$. At an arbitrary input x_0 the prediction is $\hat{y}(x_0) = x_0^T \hat{\beta}$. The entire fitted surface is characterized by the p parameters $\hat{\beta}$. Intuitively, it seems that we do not need a very large data set to fit such a model.

Let's look at an example of the linear model in a classification context. Figure 2.1 shows a scatterplot of training data on a pair of inputs X_1 and X_2 . The data are simulated, and for the present the simulation model is not important. The output class variable G has the values **BLUE** or **ORANGE**, and is represented as such in the scatterplot. There are 100 points in each of the two classes. The linear regression model was fit to these data, with the response Y coded as 0 for **BLUE** and 1 for **ORANGE**. The fitted values $\hat{Y}$ are converted to a fitted class variable $\hat{G}$ according to the rule

$$\hat{G} = \begin{cases} \text{ORANGE} & \text{if } \hat{Y} > 0.5, \\ \text{BLUE} & \text{if } \hat{Y} \leq 0.5. \end{cases} \quad (2.7)$$



FIGURE 2.1. A classification example in two dimensions. The classes are coded as a binary variable (BLUE = 0, ORANGE = 1), and then fit by linear regression. The line is the decision boundary defined by $x^T \hat{\beta} = 0.5$. The orange shaded region denotes that part of input space classified as ORANGE, while the blue region is classified as BLUE.

The set of points in $\mathbb{R}^2$ classified as ORANGE corresponds to $\{x : x^T \hat{\beta} > 0.5\}$, indicated in Figure 2.1, and the two predicted classes are separated by the decision boundary $\{x : x^T \hat{\beta} = 0.5\}$, which is linear in this case. We see that for these data there are several misclassifications on both sides of the decision boundary. Perhaps our linear model is too rigid—or are such errors unavoidable? Remember that these are errors on the training data itself, and we have not said where the constructed data came from. Consider the two possible scenarios:

Scenario 1: The training data in each class were generated from bivariate Gaussian distributions with uncorrelated components and different means.

Scenario 2: The training data in each class came from a mixture of 10 low-variance Gaussian distributions, with individual means themselves distributed as Gaussian.

A mixture of Gaussians is best described in terms of the generative model. One first generates a discrete variable that determines which of

the component Gaussians to use, and then generates an observation from the chosen density. In the case of one Gaussian per class, we will see in Chapter 4 that a linear decision boundary is the best one can do, and that our estimate is almost optimal. The region of overlap is inevitable, and future data to be predicted will be plagued by this overlap as well.

In the case of mixtures of tightly clustered Gaussians the story is different. A linear decision boundary is unlikely to be optimal, and in fact is not. The optimal decision boundary is nonlinear and disjoint, and as such will be much more difficult to obtain.

We now look at another classification and regression procedure that is in some sense at the opposite end of the spectrum to the linear model, and far better suited to the second scenario.

2.3.2 Nearest-Neighbor Methods

Nearest-neighbor methods use those observations in the training set $\mathcal{T}$ closest in input space to x to form $\hat{Y}$. Specifically, the k -nearest neighbor fit for $\hat{Y}$ is defined as follows:

$$\hat{Y}(x) = \frac{1}{k} \sum_{x_i \in N_k(x)} y_i, \quad (2.8)$$

where $N_k(x)$ is the neighborhood of x defined by the k closest points x_i in the training sample. Closeness implies a metric, which for the moment we assume is Euclidean distance. So, in words, we find the k observations with x_i closest to x in input space, and average their responses.

In Figure 2.2 we use the same training data as in Figure 2.1, and use 15-nearest-neighbor averaging of the binary coded response as the method of fitting. Thus $\hat{Y}$ is the proportion of **ORANGE**'s in the neighborhood, and so assigning class **ORANGE** to $\hat{G}$ if $\hat{Y} > 0.5$ amounts to a majority vote in the neighborhood. The colored regions indicate all those points in input space classified as **BLUE** or **ORANGE** by such a rule, in this case found by evaluating the procedure on a fine grid in input space. We see that the decision boundaries that separate the **BLUE** from the **ORANGE** regions are far more irregular, and respond to local clusters where one class dominates.

Figure 2.3 shows the results for 1-nearest-neighbor classification: $\hat{Y}$ is assigned the value y_ℓ of the closest point x_ℓ to x in the training data. In this case the regions of classification can be computed relatively easily, and correspond to a *Voronoi tessellation* of the training data. Each point x_i has an associated tile bounding the region for which it is the closest input point. For all points x in the tile, $\hat{G}(x) = g_i$. The decision boundary is even more irregular than before.

The method of k -nearest-neighbor averaging is defined in exactly the same way for regression of a quantitative output Y , although $k = 1$ would be an unlikely choice.

15-Nearest Neighbor Classifier

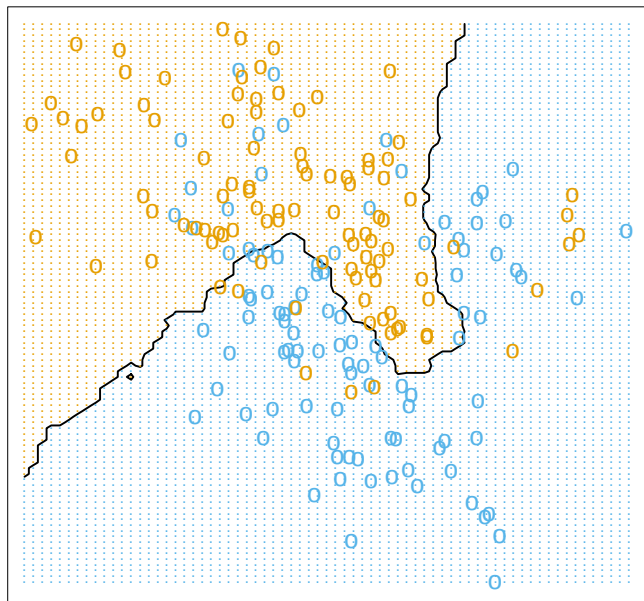


FIGURE 2.2. The same classification example in two dimensions as in Figure 2.1. The classes are coded as a binary variable (BLUE = 0, ORANGE = 1) and then fit by 15-nearest-neighbor averaging as in (2.8). The predicted class is hence chosen by majority vote amongst the 15-nearest neighbors.

In Figure 2.2 we see that far fewer training observations are misclassified than in Figure 2.1. This should not give us too much comfort, though, since in Figure 2.3 *none* of the training data are misclassified. A little thought suggests that for k -nearest-neighbor fits, the error on the training data should be approximately an increasing function of k , and will always be 0 for $k = 1$. An independent test set would give us a more satisfactory means for comparing the different methods.

It appears that k -nearest-neighbor fits have a single parameter, the number of neighbors k , compared to the p parameters in least-squares fits. Although this is the case, we will see that the *effective* number of parameters of k -nearest neighbors is N/k and is generally bigger than p , and decreases with increasing k . To get an idea of why, note that if the neighborhoods were nonoverlapping, there would be N/k neighborhoods and we would fit one parameter (a mean) in each neighborhood.

It is also clear that we cannot use sum-of-squared errors on the training set as a criterion for picking k , since we would always pick $k = 1$! It would seem that k -nearest-neighbor methods would be more appropriate for the mixture Scenario 2 described above, while for Gaussian data the decision boundaries of k -nearest neighbors would be unnecessarily noisy.

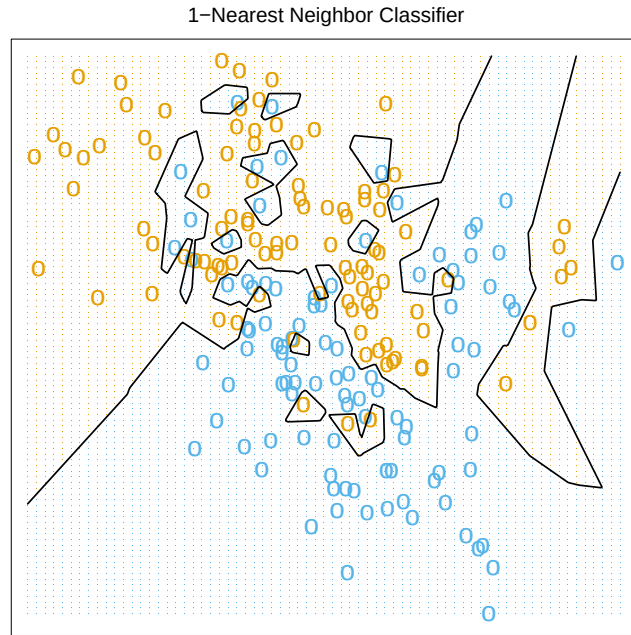


FIGURE 2.3. The same classification example in two dimensions as in Figure 2.1. The classes are coded as a binary variable (BLUE = 0, ORANGE = 1), and then predicted by 1-nearest-neighbor classification.

2.3.3 From Least Squares to Nearest Neighbors

The linear decision boundary from least squares is very smooth, and apparently stable to fit. It does appear to rely heavily on the assumption that a linear decision boundary is appropriate. In language we will develop shortly, it has low variance and potentially high bias.

On the other hand, the k -nearest-neighbor procedures do not appear to rely on any stringent assumptions about the underlying data, and can adapt to any situation. However, any particular subregion of the decision boundary depends on a handful of input points and their particular positions, and is thus wiggly and unstable—high variance and low bias.

Each method has its own situations for which it works best; in particular linear regression is more appropriate for Scenario 1 above, while nearest neighbors are more suitable for Scenario 2. The time has come to expose the oracle! The data in fact were simulated from a model somewhere between the two, but closer to Scenario 2. First we generated 10 means m_k from a bivariate Gaussian distribution $N((1, 0)^T, \mathbf{I})$ and labeled this class BLUE. Similarly, 10 more were drawn from $N((0, 1)^T, \mathbf{I})$ and labeled class ORANGE. Then for each class we generated 100 observations as follows: for each observation, we picked an m_k at random with probability $1/10$, and

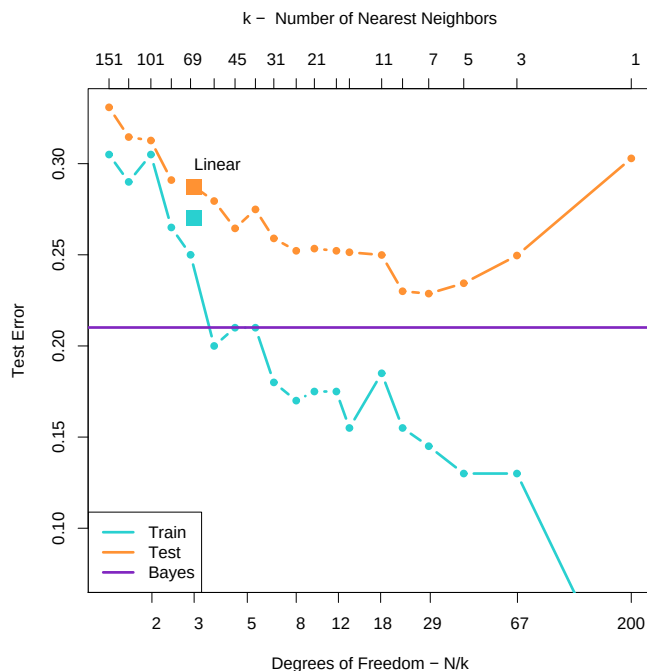


FIGURE 2.4. Misclassification curves for the simulation example used in Figures 2.1, 2.2 and 2.3. A single training sample of size 200 was used, and a test sample of size 10,000. The orange curves are test and the blue are training error for k -nearest-neighbor classification. The results for linear regression are the bigger orange and blue squares at three degrees of freedom. The purple line is the optimal Bayes error rate.

then generated a $N(m_k, \mathbf{I}/5)$, thus leading to a mixture of Gaussian clusters for each class. Figure 2.4 shows the results of classifying 10,000 new observations generated from the model. We compare the results for least squares and those for k -nearest neighbors for a range of values of k .

A large subset of the most popular techniques in use today are variants of these two simple procedures. In fact 1-nearest-neighbor, the simplest of all, captures a large percentage of the market for low-dimensional problems. The following list describes some ways in which these simple procedures have been enhanced:

- Kernel methods use weights that decrease smoothly to zero with distance from the target point, rather than the effective 0/1 weights used by k -nearest neighbors.
- In high-dimensional spaces the distance kernels are modified to emphasize some variable more than others.

- Local regression fits linear models by locally weighted least squares, rather than fitting constants locally.
- Linear models fit to a basis expansion of the original inputs allow arbitrarily complex models.
- Projection pursuit and neural network models consist of sums of non-linearly transformed linear models.

2.4 Statistical Decision Theory

In this section we develop a small amount of theory that provides a framework for developing models such as those discussed informally so far. We first consider the case of a quantitative output, and place ourselves in the world of random variables and probability spaces. Let $X \in \mathbb{R}^p$ denote a real valued random input vector, and $Y \in \mathbb{R}$ a real valued random output variable, with joint distribution $\Pr(X, Y)$. We seek a function $f(X)$ for predicting Y given values of the input X . This theory requires a *loss function* $L(Y, f(X))$ for penalizing errors in prediction, and by far the most common and convenient is *squared error loss*: $L(Y, f(X)) = (Y - f(X))^2$. This leads us to a criterion for choosing f ,

$$\text{EPE}(f) = \mathbb{E}(Y - f(X))^2 \quad (2.9)$$

$$= \int [y - f(x)]^2 \Pr(dx, dy), \quad (2.10)$$

the expected (squared) prediction error. By conditioning¹ on X , we can write EPE as

$$\text{EPE}(f) = \mathbb{E}_X \mathbb{E}_{Y|X} ([Y - f(X)]^2 | X) \quad (2.11)$$

and we see that it suffices to minimize EPE pointwise:

$$f(x) = \operatorname{argmin}_c \mathbb{E}_{Y|X} ([Y - c]^2 | X = x). \quad (2.12)$$

The solution is

$$f(x) = \mathbb{E}(Y | X = x), \quad (2.13)$$

the conditional expectation, also known as the *regression* function. Thus the best prediction of Y at any point $X = x$ is the conditional mean, when best is measured by average squared error.

The nearest-neighbor methods attempt to directly implement this recipe using the training data. At each point x , we might ask for the average of all

¹Conditioning here amounts to factoring the joint density $\Pr(X, Y) = \Pr(Y|X)\Pr(X)$ where $\Pr(Y|X) = \Pr(Y, X)/\Pr(X)$, and splitting up the bivariate integral accordingly.

those y_i s with input $x_i = x$. Since there is typically at most one observation at any point x , we settle for

$$\hat{f}(x) = \text{Ave}(y_i | x_i \in N_k(x)), \quad (2.14)$$

where “Ave” denotes average, and $N_k(x)$ is the neighborhood containing the k points in $\mathcal{T}$ closest to x . Two approximations are happening here:

- expectation is approximated by averaging over sample data;
- conditioning at a point is relaxed to conditioning on some region “close” to the target point.

For large training sample size N , the points in the neighborhood are likely to be close to x , and as k gets large the average will get more stable. In fact, under mild regularity conditions on the joint probability distribution $\Pr(X, Y)$, one can show that as $N, k \rightarrow \infty$ such that $k/N \rightarrow 0$, $\hat{f}(x) \rightarrow E(Y|X = x)$. In light of this, why look further, since it seems we have a universal approximator? We often do not have very large samples. If the linear or some more structured model is appropriate, then we can usually get a more stable estimate than k -nearest neighbors, although such knowledge has to be learned from the data as well. There are other problems though, sometimes disastrous. In Section 2.5 we see that as the dimension p gets large, so does the metric size of the k -nearest neighborhood. So settling for nearest neighborhood as a surrogate for conditioning will fail us miserably. The convergence above still holds, but the *rate* of convergence decreases as the dimension increases.

How does linear regression fit into this framework? The simplest explanation is that one assumes that the regression function $f(x)$ is approximately linear in its arguments:

$$f(x) \approx x^T \beta. \quad (2.15)$$

This is a model-based approach—we specify a model for the regression function. Plugging this linear model for $f(x)$ into EPE (2.9) and differentiating we can solve for β theoretically:

$$\beta = [E(XX^T)]^{-1}E(XY). \quad (2.16)$$

Note we have *not* conditioned on X ; rather we have used our knowledge of the functional relationship to *pool* over values of X . The least squares solution (2.6) amounts to replacing the expectation in (2.16) by averages over the training data.

So both k -nearest neighbors and least squares end up approximating conditional expectations by averages. But they differ dramatically in terms of model assumptions:

- Least squares assumes $f(x)$ is well approximated by a globally linear function.

- k -nearest neighbors assumes $f(x)$ is well approximated by a locally constant function.

Although the latter seems more palatable, we have already seen that we may pay a price for this flexibility.

Many of the more modern techniques described in this book are model based, although far more flexible than the rigid linear model. For example, additive models assume that

$$f(X) = \sum_{j=1}^p f_j(X_j). \quad (2.17)$$

This retains the additivity of the linear model, but each coordinate function f_j is arbitrary. It turns out that the optimal estimate for the additive model uses techniques such as k -nearest neighbors to approximate *univariate* conditional expectations *simultaneously* for each of the coordinate functions. Thus the problems of estimating a conditional expectation in high dimensions are swept away in this case by imposing some (often unrealistic) model assumptions, in this case additivity.

Are we happy with the criterion (2.11)? What happens if we replace the L_2 loss function with the L_1 : $E|Y - f(X)|$? The solution in this case is the conditional median,

$$\hat{f}(x) = \text{median}(Y|X = x), \quad (2.18)$$

which is a different measure of location, and its estimates are more robust than those for the conditional mean. L_1 criteria have discontinuities in their derivatives, which have hindered their widespread use. Other more resistant loss functions will be mentioned in later chapters, but squared error is analytically convenient and the most popular.

What do we do when the output is a categorical variable G ? The same paradigm works here, except we need a different loss function for penalizing prediction errors. An estimate $\hat{G}$ will assume values in $\mathcal{G}$, the set of possible classes. Our loss function can be represented by a $K \times K$ matrix $\mathbf{L}$, where $K = \text{card}(\mathcal{G})$. $\mathbf{L}$ will be zero on the diagonal and nonnegative elsewhere, where $L(k, \ell)$ is the price paid for classifying an observation belonging to class $\mathcal{G}_k$ as $\mathcal{G}_\ell$. Most often we use the *zero-one* loss function, where all misclassifications are charged a single unit. The expected prediction error is

$$\text{EPE} = E[L(G, \hat{G}(X))], \quad (2.19)$$

where again the expectation is taken with respect to the joint distribution $\text{Pr}(G, X)$. Again we condition, and can write EPE as

$$\text{EPE} = E_X \sum_{k=1}^K L[\mathcal{G}_k, \hat{G}(X)] \text{Pr}(\mathcal{G}_k|X) \quad (2.20)$$

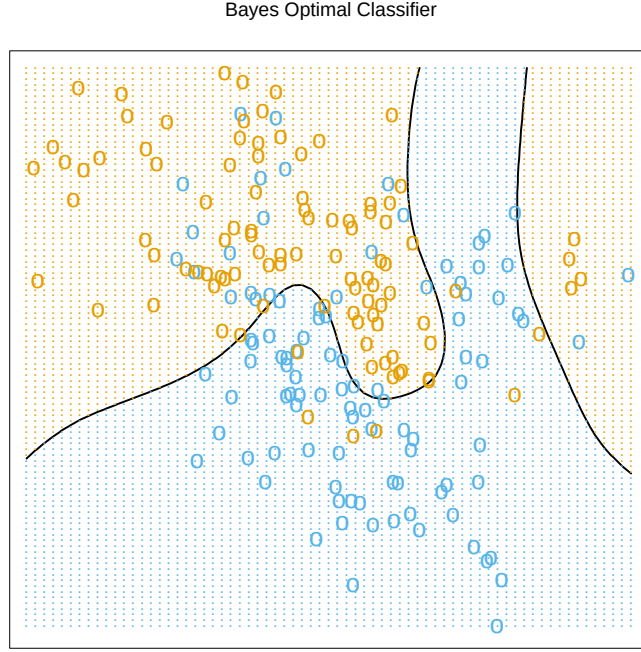


FIGURE 2.5. The optimal Bayes decision boundary for the simulation example of Figures 2.1, 2.2 and 2.3. Since the generating density is known for each class, this boundary can be calculated exactly (Exercise 2.2).

and again it suffices to minimize EPE pointwise:

$$\hat{G}(x) = \operatorname{argmin}_{g \in \mathcal{G}} \sum_{k=1}^K L(\mathcal{G}_k, g) \Pr(\mathcal{G}_k | X = x). \quad (2.21)$$

With the 0–1 loss function this simplifies to

$$\hat{G}(x) = \operatorname{argmin}_{g \in \mathcal{G}} [1 - \Pr(g | X = x)] \quad (2.22)$$

or simply

$$\hat{G}(x) = \mathcal{G}_k \text{ if } \Pr(\mathcal{G}_k | X = x) = \max_{g \in \mathcal{G}} \Pr(g | X = x). \quad (2.23)$$

This reasonable solution is known as the *Bayes classifier*, and says that we classify to the most probable class, using the conditional (discrete) distribution $\Pr(G|X)$. Figure 2.5 shows the Bayes-optimal decision boundary for our simulation example. The error rate of the Bayes classifier is called the *Bayes rate*.

Again we see that the k -nearest neighbor classifier directly approximates this solution—a majority vote in a nearest neighborhood amounts to exactly this, except that conditional probability at a point is relaxed to conditional probability within a neighborhood of a point, and probabilities are estimated by training-sample proportions.

Suppose for a two-class problem we had taken the dummy-variable approach and coded G via a binary Y , followed by squared error loss estimation. Then $\hat{f}(X) = E(Y|X) = \Pr(G = \mathcal{G}_1|X)$ if $\mathcal{G}_1$ corresponded to $Y = 1$. Likewise for a K -class problem, $E(Y_k|X) = \Pr(G = \mathcal{G}_k|X)$. This shows that our dummy-variable regression procedure, followed by classification to the largest fitted value, is another way of representing the Bayes classifier. Although this theory is exact, in practice problems can occur, depending on the regression model used. For example, when linear regression is used, $\hat{f}(X)$ need not be positive, and we might be suspicious about using it as an estimate of a probability. We will discuss a variety of approaches to modeling $\Pr(G|X)$ in Chapter 4.

2.5 Local Methods in High Dimensions

We have examined two learning techniques for prediction so far: the stable but biased linear model and the less stable but apparently less biased class of k -nearest-neighbor estimates. It would seem that with a reasonably large set of training data, we could always approximate the theoretically optimal conditional expectation by k -nearest-neighbor averaging, since we should be able to find a fairly large neighborhood of observations close to any x and average them. This approach and our intuition breaks down in high dimensions, and the phenomenon is commonly referred to as the *curse of dimensionality* (Bellman, 1961). There are many manifestations of this problem, and we will examine a few here.

Consider the nearest-neighbor procedure for inputs uniformly distributed in a p -dimensional unit hypercube, as in Figure 2.6. Suppose we send out a hypercubical neighborhood about a target point to capture a fraction r of the observations. Since this corresponds to a fraction r of the unit volume, the expected edge length will be $e_p(r) = r^{1/p}$. In ten dimensions $e_{10}(0.01) = 0.63$ and $e_{10}(0.1) = 0.80$, while the entire range for each input is only 1.0. So to capture 1% or 10% of the data to form a local average, we must cover 63% or 80% of the range of each input variable. Such neighborhoods are no longer “local.” Reducing r dramatically does not help much either, since the fewer observations we average, the higher is the variance of our fit.

Another consequence of the sparse sampling in high dimensions is that all sample points are close to an edge of the sample. Consider N data points uniformly distributed in a p -dimensional unit ball centered at the origin. Suppose we consider a nearest-neighbor estimate at the origin. The median

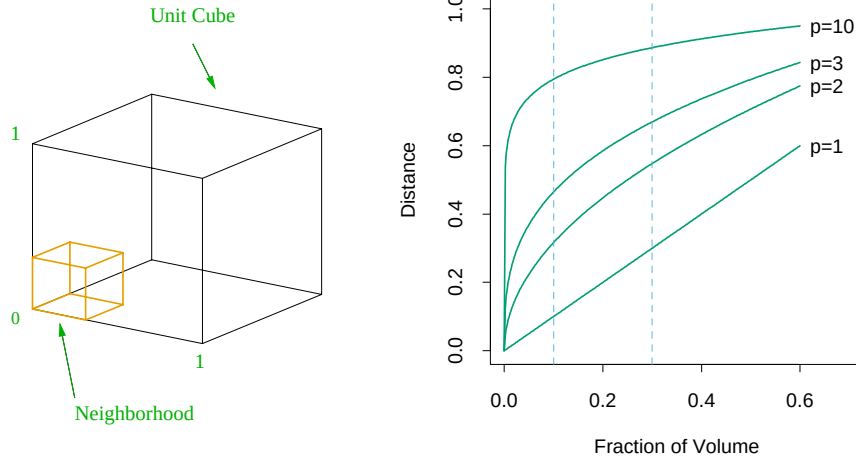


FIGURE 2.6. The curse of dimensionality is well illustrated by a subcubical neighborhood for uniform data in a unit cube. The figure on the right shows the side-length of the subcube needed to capture a fraction r of the volume of the data, for different dimensions p . In ten dimensions we need to cover 80% of the range of each coordinate to capture 10% of the data.

distance from the origin to the closest data point is given by the expression

$$d(p, N) = \left(1 - \frac{1}{2}^{1/N}\right)^{1/p} \quad (2.24)$$

(Exercise 2.3). A more complicated expression exists for the mean distance to the closest point. For $N = 500$, $p = 10$, $d(p, N) \approx 0.52$, more than halfway to the boundary. Hence most data points are closer to the boundary of the sample space than to any other data point. The reason that this presents a problem is that prediction is much more difficult near the edges of the training sample. One must extrapolate from neighboring sample points rather than interpolate between them.

Another manifestation of the curse is that the sampling density is proportional to $N^{1/p}$, where p is the dimension of the input space and N is the sample size. Thus, if $N_1 = 100$ represents a dense sample for a single input problem, then $N_{10} = 100^{10}$ is the sample size required for the same sampling density with 10 inputs. Thus in high dimensions all feasible training samples sparsely populate the input space.

Let us construct another uniform example. Suppose we have 1000 training examples x_i generated uniformly on $[-1, 1]^p$. Assume that the true relationship between X and Y is

$$Y = f(X) = e^{-8\|X\|^2},$$

without any measurement error. We use the 1-nearest-neighbor rule to predict y_0 at the test-point $x_0 = 0$. Denote the training set by $\mathcal{T}$. We can

compute the expected prediction error at x_0 for our procedure, averaging over all such samples of size 1000. Since the problem is deterministic, this is the mean squared error (MSE) for estimating $f(0)$:

$$\begin{aligned} \text{MSE}(x_0) &= \mathbb{E}_{\mathcal{T}}[f(x_0) - \hat{y}_0]^2 \\ &= \mathbb{E}_{\mathcal{T}}[\hat{y}_0 - \mathbb{E}_{\mathcal{T}}(\hat{y}_0)]^2 + [\mathbb{E}_{\mathcal{T}}(\hat{y}_0) - f(x_0)]^2 \\ &= \text{Var}_{\mathcal{T}}(\hat{y}_0) + \text{Bias}^2(\hat{y}_0). \end{aligned} \quad (2.25)$$

Figure 2.7 illustrates the setup. We have broken down the MSE into two components that will become familiar as we proceed: variance and squared bias. Such a decomposition is always possible and often useful, and is known as the *bias–variance decomposition*. Unless the nearest neighbor is at 0, $\hat{y}_0$ will be smaller than $f(0)$ in this example, and so the average estimate will be biased downward. The variance is due to the sampling variance of the 1-nearest neighbor. In low dimensions and with $N = 1000$, the nearest neighbor is very close to 0, and so both the bias and variance are small. As the dimension increases, the nearest neighbor tends to stray further from the target point, and both bias and variance are incurred. By $p = 10$, for more than 99% of the samples the nearest neighbor is a distance greater than 0.5 from the origin. Thus as p increases, the estimate tends to be 0 more often than not, and hence the MSE levels off at 1.0, as does the bias, and the variance starts dropping (an artifact of this example).

Although this is a highly contrived example, similar phenomena occur more generally. The complexity of functions of many variables can grow exponentially with the dimension, and if we wish to be able to estimate such functions with the same accuracy as function in low dimensions, then we need the size of our training set to grow exponentially as well. In this example, the function is a complex interaction of all p variables involved.

The dependence of the bias term on distance depends on the truth, and it need not always dominate with 1-nearest neighbor. For example, if the function always involves only a few dimensions as in Figure 2.8, then the variance can dominate instead.

Suppose, on the other hand, that we know that the relationship between Y and X is linear,

$$Y = X^T \beta + \varepsilon, \quad (2.26)$$

where $\varepsilon \sim N(0, \sigma^2)$ and we fit the model by least squares to the training data. For an arbitrary test point x_0 , we have $\hat{y}_0 = x_0^T \hat{\beta}$, which can be written as $\hat{y}_0 = x_0^T \beta + \sum_{i=1}^N \ell_i(x_0) \varepsilon_i$, where $\ell_i(x_0)$ is the i th element of $\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} x_0$. Since under this model the least squares estimates are

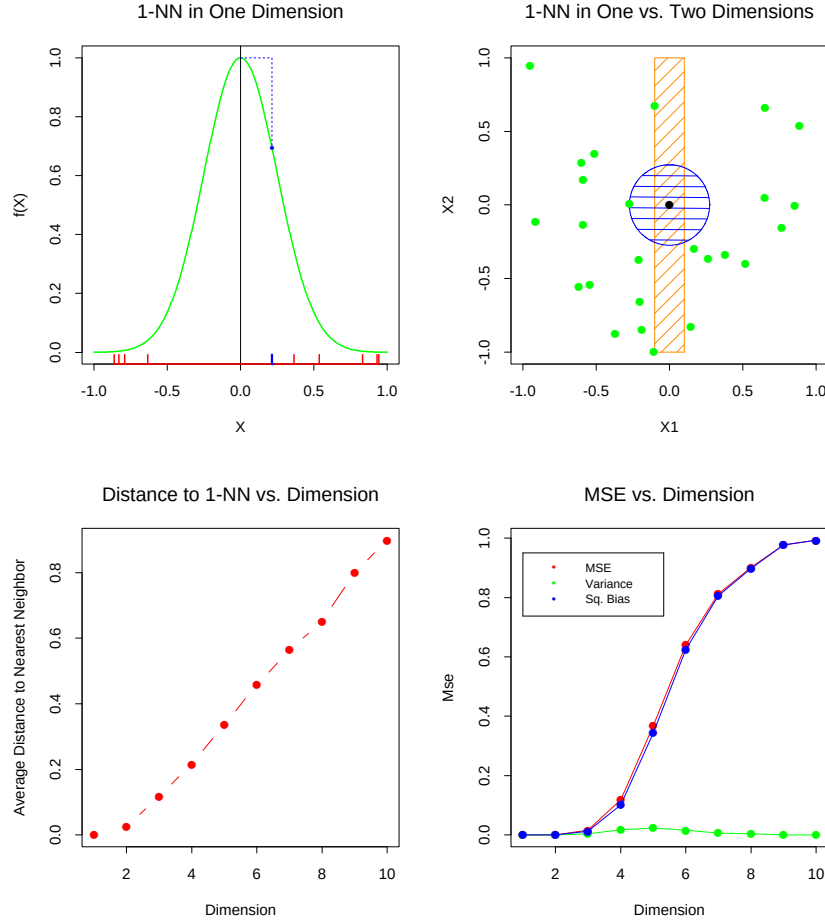


FIGURE 2.7. A simulation example, demonstrating the curse of dimensionality and its effect on MSE, bias and variance. The input features are uniformly distributed in $[-1, 1]^p$ for $p = 1, \dots, 10$. The top left panel shows the target function (no noise) in $\mathbb{R}$: $f(X) = e^{-8\|X\|^2}$, and demonstrates the error that 1-nearest neighbor makes in estimating $f(0)$. The training point is indicated by the blue tick mark. The top right panel illustrates why the radius of the 1-nearest neighborhood increases with dimension p . The lower left panel shows the average radius of the 1-nearest neighborhoods. The lower-right panel shows the MSE, squared bias and variance curves as a function of dimension p .

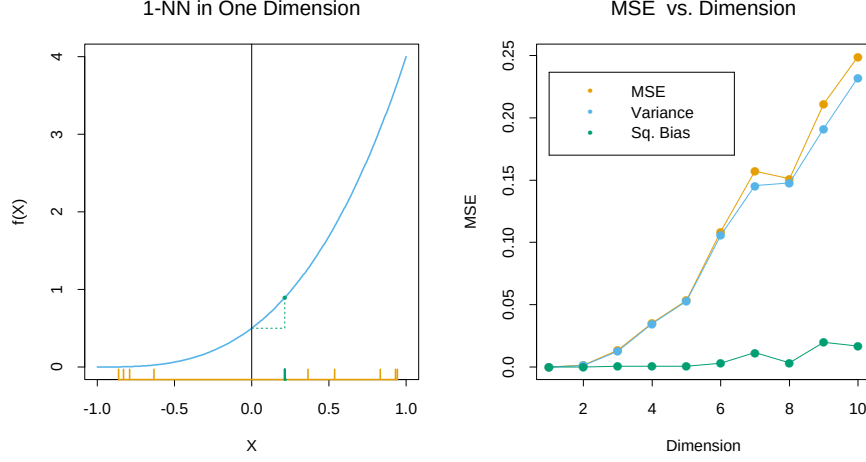


FIGURE 2.8. A simulation example with the same setup as in Figure 2.7. Here the function is constant in all but one dimension: $f(X) = \frac{1}{2}(X_1 + 1)^3$. The variance dominates.

unbiased, we find that

$$\begin{aligned}
 \text{EPE}(x_0) &= \mathbb{E}_{y_0|x_0} \mathbb{E}_{\mathcal{T}} (y_0 - \hat{y}_0)^2 \\
 &= \text{Var}(y_0|x_0) + \mathbb{E}_{\mathcal{T}} [\hat{y}_0 - \mathbb{E}_{\mathcal{T}} \hat{y}_0]^2 + [\mathbb{E}_{\mathcal{T}} \hat{y}_0 - x_0^T \beta]^2 \\
 &= \text{Var}(y_0|x_0) + \text{Var}_{\mathcal{T}}(\hat{y}_0) + \text{Bias}^2(\hat{y}_0) \\
 &= \sigma^2 + \mathbb{E}_{\mathcal{T}} x_0^T (\mathbf{X}^T \mathbf{X})^{-1} x_0 \sigma^2 + 0^2.
 \end{aligned} \tag{2.27}$$

Here we have incurred an additional variance σ^2 in the prediction error, since our target is not deterministic. There is no bias, and the variance depends on x_0 . If N is large and $\mathcal{T}$ were selected at random, and assuming $\mathbb{E}(X) = 0$, then $\mathbf{X}^T \mathbf{X} \rightarrow N \text{Cov}(X)$ and

$$\begin{aligned}
 \mathbb{E}_{x_0} \text{EPE}(x_0) &\sim \mathbb{E}_{x_0} x_0^T \text{Cov}(X)^{-1} x_0 \sigma^2 / N + \sigma^2 \\
 &= \text{trace}[\text{Cov}(X)^{-1} \text{Cov}(x_0)] \sigma^2 / N + \sigma^2 \\
 &= \sigma^2(p/N) + \sigma^2.
 \end{aligned} \tag{2.28}$$

Here we see that the expected EPE increases linearly as a function of p , with slope σ^2/N . If N is large and/or σ^2 is small, this growth in variance is negligible (0 in the deterministic case). By imposing some heavy restrictions on the class of models being fitted, we have avoided the curse of dimensionality. Some of the technical details in (2.27) and (2.28) are derived in Exercise 2.5.

Figure 2.9 compares 1-nearest neighbor vs. least squares in two situations, both of which have the form $Y = f(X) + \varepsilon$, X uniform as before, and $\varepsilon \sim N(0, 1)$. The sample size is $N = 500$. For the orange curve, $f(x)$

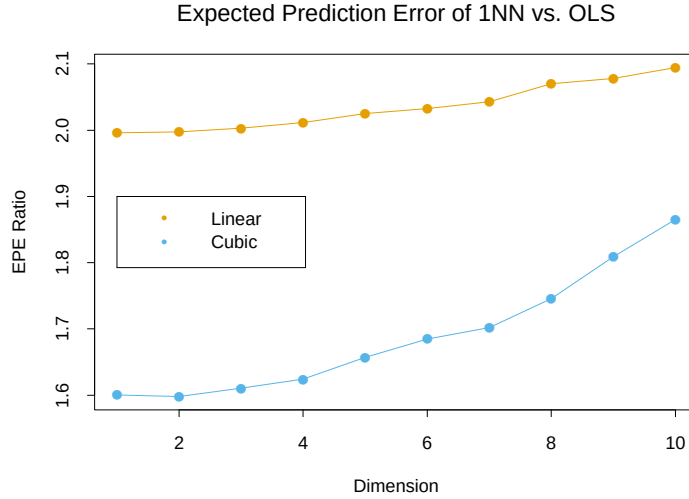


FIGURE 2.9. The curves show the expected prediction error (at $x_0 = 0$) for 1-nearest neighbor relative to least squares for the model $Y = f(X) + \varepsilon$. For the orange curve, $f(x) = x_1$, while for the blue curve $f(x) = \frac{1}{2}(x_1 + 1)^3$.

is linear in the first coordinate, for the blue curve, cubic as in Figure 2.8. Shown is the relative EPE of 1-nearest neighbor to least squares, which appears to start at around 2 for the linear case. Least squares is unbiased in this case, and as discussed above the EPE is slightly above $\sigma^2 = 1$. The EPE for 1-nearest neighbor is always above 2, since the variance of $\hat{f}(x_0)$ in this case is at least σ^2 , and the ratio increases with dimension as the nearest neighbor strays from the target point. For the cubic case, least squares is biased, which moderates the ratio. Clearly we could manufacture examples where the bias of least squares would dominate the variance, and the 1-nearest neighbor would come out the winner.

By relying on rigid assumptions, the linear model has no bias at all and negligible variance, while the error in 1-nearest neighbor is substantially larger. However, if the assumptions are wrong, all bets are off and the 1-nearest neighbor may dominate. We will see that there is a whole spectrum of models between the rigid linear models and the extremely flexible 1-nearest-neighbor models, each with their own assumptions and biases, which have been proposed specifically to avoid the exponential growth in complexity of functions in high dimensions by drawing heavily on these assumptions.

Before we delve more deeply, let us elaborate a bit on the concept of *statistical models* and see how they fit into the prediction framework.

2.6 Statistical Models, Supervised Learning and Function Approximation

Our goal is to find a useful approximation $\hat{f}(x)$ to the function $f(x)$ that underlies the predictive relationship between the inputs and outputs. In the theoretical setting of Section 2.4, we saw that squared error loss lead us to the regression function $f(x) = E(Y|X = x)$ for a quantitative response. The class of nearest-neighbor methods can be viewed as direct estimates of this conditional expectation, but we have seen that they can fail in at least two ways:

- if the dimension of the input space is high, the nearest neighbors need not be close to the target point, and can result in large errors;
- if special structure is known to exist, this can be used to reduce both the bias and the variance of the estimates.

We anticipate using other classes of models for $f(x)$, in many cases specifically designed to overcome the dimensionality problems, and here we discuss a framework for incorporating them into the prediction problem.

2.6.1 A Statistical Model for the Joint Distribution $\Pr(X, Y)$

Suppose in fact that our data arose from a statistical model

$$Y = f(X) + \varepsilon, \quad (2.29)$$

where the random error ε has $E(\varepsilon) = 0$ and is independent of X . Note that for this model, $f(x) = E(Y|X = x)$, and in fact the conditional distribution $\Pr(Y|X)$ depends on X *only* through the conditional mean $f(x)$.

The additive error model is a useful approximation to the truth. For most systems the input–output pairs (X, Y) will not have a deterministic relationship $Y = f(X)$. Generally there will be other unmeasured variables that also contribute to Y , including measurement error. The additive model assumes that we can capture all these departures from a deterministic relationship via the error ε .

For some problems a deterministic relationship does hold. Many of the classification problems studied in machine learning are of this form, where the response surface can be thought of as a colored map defined in $\mathbb{R}^p$. The training data consist of colored examples from the map $\{x_i, g_i\}$, and the goal is to be able to color any point. Here the function is deterministic, and the randomness enters through the x location of the training points. For the moment we will not pursue such problems, but will see that they can be handled by techniques appropriate for the error-based models.

The assumption in (2.29) that the errors are independent and identically distributed is not strictly necessary, but seems to be at the back of our mind

when we average squared errors uniformly in our EPE criterion. With such a model it becomes natural to use least squares as a data criterion for model estimation as in (2.1). Simple modifications can be made to avoid the independence assumption; for example, we can have $\text{Var}(Y|X = x) = \sigma(x)$, and now both the mean and variance depend on X . In general the conditional distribution $\text{Pr}(Y|X)$ can depend on X in complicated ways, but the additive error model precludes these.

So far we have concentrated on the quantitative response. Additive error models are typically not used for qualitative outputs G ; in this case the target function $p(X)$ is the conditional density $\text{Pr}(G|X)$, and this is modeled directly. For example, for two-class data, it is often reasonable to assume that the data arise from independent binary trials, with the probability of one particular outcome being $p(X)$, and the other $1 - p(X)$. Thus if Y is the 0–1 coded version of G , then $E(Y|X = x) = p(x)$, but the variance depends on x as well: $\text{Var}(Y|X = x) = p(x)[1 - p(x)]$.

2.6.2 Supervised Learning

Before we launch into more statistically oriented jargon, we present the function-fitting paradigm from a machine learning point of view. Suppose for simplicity that the errors are additive and that the model $Y = f(X) + \varepsilon$ is a reasonable assumption. Supervised learning attempts to learn f by example through a *teacher*. One observes the system under study, both the inputs and outputs, and assembles a *training* set of observations $\mathcal{T} = (x_i, y_i)$, $i = 1, \dots, N$. The observed input values to the system x_i are also fed into an artificial system, known as a learning algorithm (usually a computer program), which also produces outputs $\hat{f}(x_i)$ in response to the inputs. The learning algorithm has the property that it can modify its input/output relationship $\hat{f}$ in response to differences $y_i - \hat{f}(x_i)$ between the original and generated outputs. This process is known as *learning by example*. Upon completion of the learning process the hope is that the artificial and real outputs will be close enough to be useful for all sets of inputs likely to be encountered in practice.

2.6.3 Function Approximation

The learning paradigm of the previous section has been the motivation for research into the supervised learning problem in the fields of machine learning (with analogies to human reasoning) and neural networks (with biological analogies to the brain). The approach taken in applied mathematics and statistics has been from the perspective of function approximation and estimation. Here the data pairs $\{x_i, y_i\}$ are viewed as points in a $(p + 1)$ -dimensional Euclidean space. The function $f(x)$ has domain equal to the p -dimensional input subspace, and is related to the data via a model

such as $y_i = f(x_i) + \varepsilon_i$. For convenience in this chapter we will assume the domain is $\mathbb{R}^p$, a p -dimensional Euclidean space, although in general the inputs can be of mixed type. The goal is to obtain a useful approximation to $f(x)$ for all x in some region of $\mathbb{R}^p$, given the representations in $\mathcal{T}$. Although somewhat less glamorous than the learning paradigm, treating supervised learning as a problem in function approximation encourages the geometrical concepts of Euclidean spaces and mathematical concepts of probabilistic inference to be applied to the problem. This is the approach taken in this book.

Many of the approximations we will encounter have associated a set of parameters θ that can be modified to suit the data at hand. For example, the linear model $f(x) = x^T \beta$ has $\theta = \beta$. Another class of useful approximators can be expressed as *linear basis expansions*

$$f_\theta(x) = \sum_{k=1}^K h_k(x) \theta_k, \quad (2.30)$$

where the h_k are a suitable set of functions or transformations of the input vector x . Traditional examples are polynomial and trigonometric expansions, where for example h_k might be x_1^2 , $x_1 x_2^2$, $\cos(x_1)$ and so on. We also encounter nonlinear expansions, such as the sigmoid transformation common to neural network models,

$$h_k(x) = \frac{1}{1 + \exp(-x^T \beta_k)}. \quad (2.31)$$

We can use least squares to estimate the parameters θ in f_θ as we did for the linear model, by minimizing the residual sum-of-squares

$$\text{RSS}(\theta) = \sum_{i=1}^N (y_i - f_\theta(x_i))^2 \quad (2.32)$$

as a function of θ . This seems a reasonable criterion for an additive error model. In terms of function approximation, we imagine our parameterized function as a surface in $p + 1$ space, and what we observe are noisy realizations from it. This is easy to visualize when $p = 2$ and the vertical coordinate is the output y , as in Figure 2.10. The noise is in the output coordinate, so we find the set of parameters such that the fitted surface gets as close to the observed points as possible, where close is measured by the sum of squared vertical errors in $\text{RSS}(\theta)$.

For the linear model we get a simple closed form solution to the minimization problem. This is also true for the basis function methods, if the basis functions themselves do not have any hidden parameters. Otherwise the solution requires either iterative methods or numerical optimization.

While least squares is generally very convenient, it is not the only criterion used and in some cases would not make much sense. A more general

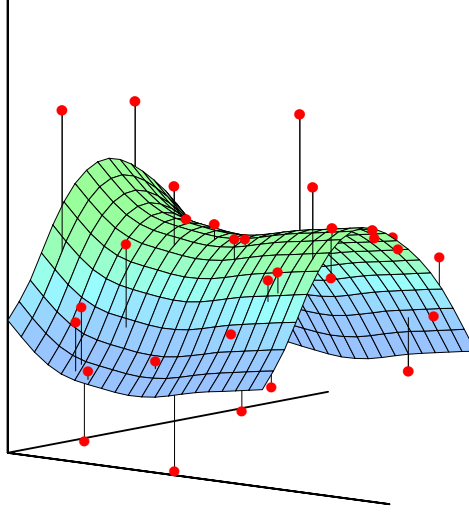


FIGURE 2.10. Least squares fitting of a function of two inputs. The parameters of $f_\theta(x)$ are chosen so as to minimize the sum-of-squared vertical errors.

principle for estimation is *maximum likelihood estimation*. Suppose we have a random sample y_i , $i = 1, \dots, N$ from a density $\Pr_\theta(y)$ indexed by some parameters θ . The log-probability of the observed sample is

$$L(\theta) = \sum_{i=1}^N \log \Pr_\theta(y_i). \quad (2.33)$$

The principle of maximum likelihood assumes that the most reasonable values for θ are those for which the probability of the observed sample is largest. Least squares for the additive error model $Y = f_\theta(X) + \varepsilon$, with $\varepsilon \sim N(0, \sigma^2)$, is equivalent to maximum likelihood using the conditional likelihood

$$\Pr(Y|X, \theta) = N(f_\theta(X), \sigma^2). \quad (2.34)$$

So although the additional assumption of normality seems more restrictive, the results are the same. The log-likelihood of the data is

$$L(\theta) = -\frac{N}{2} \log(2\pi) - N \log \sigma - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - f_\theta(x_i))^2, \quad (2.35)$$

and the only term involving θ is the last, which is $\text{RSS}(\theta)$ up to a scalar negative multiplier.

A more interesting example is the multinomial likelihood for the regression function $\Pr(G|X)$ for a qualitative output G . Suppose we have a model $\Pr(G = \mathcal{G}_k|X = x) = p_{k,\theta}(x)$, $k = 1, \dots, K$ for the conditional probability of each class given X , indexed by the parameter vector θ . Then the

log-likelihood (also referred to as the cross-entropy) is

$$L(\theta) = \sum_{i=1}^N \log p_{g_i, \theta}(x_i), \quad (2.36)$$

and when maximized it delivers values of θ that best conform with the data in this likelihood sense.

2.7 Structured Regression Models

We have seen that although nearest-neighbor and other local methods focus directly on estimating the function at a point, they face problems in high dimensions. They may also be inappropriate even in low dimensions in cases where more structured approaches can make more efficient use of the data. This section introduces classes of such structured approaches. Before we proceed, though, we discuss further the need for such classes.

2.7.1 *Difficulty of the Problem*

Consider the RSS criterion for an arbitrary function f ,

$$\text{RSS}(f) = \sum_{i=1}^N (y_i - f(x_i))^2. \quad (2.37)$$

Minimizing (2.37) leads to infinitely many solutions: any function $\hat{f}$ passing through the training points (x_i, y_i) is a solution. Any particular solution chosen might be a poor predictor at test points different from the training points. If there are multiple observation pairs $x_i, y_{i\ell}$, $\ell = 1, \dots, N_i$ at each value of x_i , the risk is limited. In this case, the solutions pass through the average values of the $y_{i\ell}$ at each x_i ; see Exercise 2.6. The situation is similar to the one we have already visited in Section 2.4; indeed, (2.37) is the finite sample version of (2.11) on page 18. If the sample size N were sufficiently large such that repeats were guaranteed and densely arranged, it would seem that these solutions might all tend to the limiting conditional expectation.

In order to obtain useful results for finite N , we must restrict the eligible solutions to (2.37) to a smaller set of functions. How to decide on the nature of the restrictions is based on considerations outside of the data. These restrictions are sometimes encoded via the parametric representation of f_θ , or may be built into the learning method itself, either implicitly or explicitly. These restricted classes of solutions are the major topic of this book. One thing should be clear, though. Any restrictions imposed on f that lead to a unique solution to (2.37) do not really remove the ambiguity

caused by the multiplicity of solutions. There are infinitely many possible restrictions, each leading to a unique solution, so the ambiguity has simply been transferred to the choice of constraint.

In general the constraints imposed by most learning methods can be described as *complexity* restrictions of one kind or another. This usually means some kind of regular behavior in small neighborhoods of the input space. That is, for all input points x sufficiently close to each other in some metric, $\hat{f}$ exhibits some special structure such as nearly constant, linear or low-order polynomial behavior. The estimator is then obtained by averaging or polynomial fitting in that neighborhood.

The strength of the constraint is dictated by the neighborhood size. The larger the size of the neighborhood, the stronger the constraint, and the more sensitive the solution is to the particular choice of constraint. For example, local constant fits in infinitesimally small neighborhoods is no constraint at all; local linear fits in very large neighborhoods is almost a globally linear model, and is very restrictive.

The nature of the constraint depends on the metric used. Some methods, such as kernel and local regression and tree-based methods, directly specify the metric and size of the neighborhood. The nearest-neighbor methods discussed so far are based on the assumption that locally the function is constant; close to a target input x_0 , the function does not change much, and so close outputs can be averaged to produce $\hat{f}(x_0)$. Other methods such as splines, neural networks and basis-function methods implicitly define neighborhoods of local behavior. In Section 5.4.1 we discuss the concept of an *equivalent kernel* (see Figure 5.8 on page 157), which describes this local dependence for any method linear in the outputs. These equivalent kernels in many cases look just like the explicitly defined weighting kernels discussed above—peaked at the target point and falling away smoothly away from it.

One fact should be clear by now. Any method that attempts to produce locally varying functions in small isotropic neighborhoods will run into problems in high dimensions—again the curse of dimensionality. And conversely, all methods that overcome the dimensionality problems have an associated—and often implicit or adaptive—metric for measuring neighborhoods, which basically does not allow the neighborhood to be simultaneously small in all directions.

2.8 Classes of Restricted Estimators

The variety of nonparametric regression techniques or learning methods fall into a number of different classes depending on the nature of the restrictions imposed. These classes are not distinct, and indeed some methods fall in several classes. Here we give a brief summary, since detailed descriptions

are given in later chapters. Each of the classes has associated with it one or more parameters, sometimes appropriately called *smoothing* parameters, that control the effective size of the local neighborhood. Here we describe three broad classes.

2.8.1 Roughness Penalty and Bayesian Methods

Here the class of functions is controlled by explicitly penalizing $\text{RSS}(f)$ with a roughness penalty

$$\text{PRSS}(f; \lambda) = \text{RSS}(f) + \lambda J(f). \quad (2.38)$$

The user-selected functional $J(f)$ will be large for functions f that vary too rapidly over small regions of input space. For example, the popular *cubic smoothing spline* for one-dimensional inputs is the solution to the penalized least-squares criterion

$$\text{PRSS}(f; \lambda) = \sum_{i=1}^N (y_i - f(x_i))^2 + \lambda \int [f''(x)]^2 dx. \quad (2.39)$$

The roughness penalty here controls large values of the second derivative of f , and the amount of penalty is dictated by $\lambda \geq 0$. For $\lambda = 0$ no penalty is imposed, and any interpolating function will do, while for $\lambda = \infty$ only functions linear in x are permitted.

Penalty functionals J can be constructed for functions in any dimension, and special versions can be created to impose special structure. For example, additive penalties $J(f) = \sum_{j=1}^p J(f_j)$ are used in conjunction with additive functions $f(X) = \sum_{j=1}^p f_j(X_j)$ to create additive models with smooth coordinate functions. Similarly, *projection pursuit regression* models have $f(X) = \sum_{m=1}^M g_m(\alpha_m^T X)$ for adaptively chosen directions α_m , and the functions g_m can each have an associated roughness penalty.

Penalty function, or *regularization* methods, express our prior belief that the type of functions we seek exhibit a certain type of smooth behavior, and indeed can usually be cast in a Bayesian framework. The penalty J corresponds to a log-prior, and $\text{PRSS}(f; \lambda)$ the log-posterior distribution, and minimizing $\text{PRSS}(f; \lambda)$ amounts to finding the posterior mode. We discuss roughness-penalty approaches in Chapter 5 and the Bayesian paradigm in Chapter 8.

2.8.2 Kernel Methods and Local Regression

These methods can be thought of as explicitly providing estimates of the regression function or conditional expectation by specifying the nature of the local neighborhood, and of the class of regular functions fitted locally. The local neighborhood is specified by a *kernel function* $K_\lambda(x_0, x)$ which assigns